1 Lesson 28-10-26

We spent the last lessons implementing a **round-robin** scheduler for our OS. This is all better explained in code than in notes.

Today we will do a theory-only lesson about scheduling algorithm The task model is the same from the previous lesson.

1.1 Fixed priority

1.1.1 RM scheduling

RM (*Rate Monotonic*) is a *fixed priority* scheduling algorithm. Fixed in this case means that each task gets a priority once, before the system starts, and keeps it forever (or until it terminates, in case of a process).

In rate monotonic, the task priority is:

$$\text{priority} = \frac{1}{t}$$

where T is the task period, with appropiate tie rules. It is *preemptive*: a released job with a shorter period preempts the running one.

1.1.2 Response-time analysis

RM scheduling is *optimal* among fixed priority scheduling algorithms.

We know how to get a sufficient (but not necessary) condition for the feasibility of RM scheduling on a set of tasks, which is from *Liu-Layland* (`https://raw.githubusercontent.com/seggiani-luca/eosa-notes/2f281524d2b6b70b26ef55339e762293d97a5d43/notes/liu-layland/main.pdf` is a sketch of the proof from my notes). The bound formula we get from it is:

$$U < n(2^{\frac{1}{n}} - 1)$$

of the utilization U for n tasks.

A techique we want to give now is **RTA** (*Response-Time Analisys*). Let R_i be the time a tasks takes to do its processing, including the interference from other tasks. This is just our response time $R = f - a$ with f the termination time and a the period edge.

We can define R_i recursively for each task:

$$R_i = C_i + \sum_{j\,:\,\text{prio}(j)>\text{prio}(i)}^{n} \left\lceil \frac{R_i}{T_j} \right\rceil \cdot C_j$$

We know that $\frac{R_i}{T_j}$ is simply the time task j is dispatched in the execution window R_i of the current task.

Then this becomes simply the time the task takes to execute by itself, plus the time all higher priority tasks "steal away" from it, every time they are scheduled in its execution window.

The way we evaluate this is by starting with $R_i(0) = C_i$ for each task. Then we update:

$$R(k+1) = C_i + \sum_{j\,:\,\text{prio}(j)>\text{prio}(i)}^{n} \left\lceil \frac{R_i(k)}{T_j} \right\rceil \cdot C_j$$

Note that the highest priority task task R_n has no interference:

$$R_n = C_n$$

so the response time is the same after the upate.

For all other tasks the update changes values, and we stop the iteration when we reach a fixed point $R_i(k+1) = R_i(k)$ for all tasks. The task set is thus schedulable when we can say:

$$R_i \leq D_i, \quad \forall i = 1, 2, ..., n$$

So each task respects its deadline.

1.2 Dynamic priority

1.2.1 EDF scheduling

Let's move on to **EDF** (*Earliest Deadline First*) scheduling. This is a *dynamic priority* scheduling algorithm, which means the priority belongs to the job, not to the task: two jobs of the same task have different deadlines. IF priorities are dynamic the ranking between two tasks can change over time.

The priority in earliest deadline first is that at every instant, we run the ready job with the earliest absolute deadline:

$$d = a + D$$

where a is the last dispatch instant.

This algorithm is preemptive too: a released job with a strictly earlier deadline preempts the running one.

The kernel overhead is that at each release the kernel must compute d and keep the ready jobs ordered by d (higher than RM).

EDG scheduling is *optimal* among dynamic priority scheduling algorithms. We have a simple schedulability bound (also from Liu-Layland):

$$U < 1$$

1.3 Aperiodic jobs

Let's now focus on **aperiodic** jobs. An aperiodic job arrives once, at a time that is not regular: a key press, a message from the network, an alarm. It has an execution time C and, often, a deadline D.

We don't know a T period, as we did before: we only know the arrival time a.

The algorithms will be:

- For **only one-shot sets**: the goal is to meet the deadlines, or be as little late as possible. Algorithms: Jackson and Horn;
- For **mixed sets**: hard periodic tasks plus aperiodic jobs whose arrival is not known in advance. The goal is to keep every periodic deadline and serve the aperiodic jobs fast. Algorithms: background, polling server, deferrable server.

The *lateness* of a task is simply $L = f - d$, that is the time past the deadline the task took to complete. Ideally we want this negative.

1.3.1 EDD scheduling

This is *Jackson*'s algoritm, known as **EDD** (*Earliest Due Deadline*).

The problem is given by the triple:

$$(1, \text{sync}, L_{\max})$$

which stands for one processor, n independent jobs that all arrive at 0, each with its own Cc and D. The goal is to minimise $L_{\max}$.

Earliest Due Date: run the jobs one after the other in order of non-decreasing deadline. All jobs are there at 0: preemption cannot help, so EDD runs each job to the end. The order is fixed before the start: sort the jobs by d.

Jackson's theorem states that EDD is optimal for Lmax: no other order gives a smaller maximum lateness . So if EDD misses a deadline ($L_{\max} > 0$), every order misses one: the set is infeasible.

1.3.2 EDF over jobs scheduling

This is *Horn*'s algoritm, known as **EDF** (*Earliest Deadline First*) over jobs.

The problem is given by the triple:

$$(1, \text{preempt}, L_{\max})$$

which stands for one processor, n independent jobs that arrive at any time, each with its own Cc and D. The goal is to minimise $L_{\max}$.

This is Horn's rule: at every instant, run the ready job with the earliest absolute deadline $d = a + D$. A new job with a strictly earlier deadline preempts the running one.

This is EDF, applied to one-shot jobs instead of periodic tasks. Horn's theorem states that EDF is optimal for $L_{\max}$: the same swap argument as Jackson, applied to the slots of the schedule. Preemption is needed: a job with a near deadline may arrive while a long job runs. Without preemption, the problem with arbitrary arrivals is much harder. With all arrivals at 0 there is no preemption, and Horn gives the same schedule as Jackson.